

Redes Neurais Convolucionais e Recorrentes Aplicadas à Classificação de Frutas e Previsão de Temperatura

1st Eduardo Caetano
ISEP

Instituto Superior de Engenharia do Porto
Porto, Portugal
1250509@isep.ipp.pt

2nd João Veríssimo
ISEP

Instituto Superior de Engenharia do Porto
Porto, Portugal
1201806@isep.ipp.pt

3rd Jorge Cruz
ISEP

Instituto Superior de Engenharia do Porto
Porto, Portugal
1221715@isep.ipp.pt

Abstract—Este trabalho aborda dois problemas clássicos da aprendizagem profunda: a classificação de imagens de frutos e a previsão de séries temporais meteorológicas.

No primeiro problema foram desenvolvidas redes neurais convolucionais (CNN) para classificar 144 categorias do conjunto de dados Fruits-360. Foram estudados, de forma sistemática, os efeitos do número de filtros, do *stride*, do tipo de *pooling* e de diferentes estratégias de regularização. Adicionalmente, comparou-se uma CNN desenvolvida de raiz com um modelo pré-treinado MobileNetV2 ajustado através de *fine-tuning*. O modelo final, baseado em MobileNetV2 com ajuste das 40 camadas superiores, alcançou uma *accuracy* de 93,96% no conjunto de teste, bem como valores de *precision* e *recall* macro iguais a 0,9513, ultrapassando confortavelmente os requisitos definidos.

No segundo problema foram treinadas redes recorrentes (*SimpleRNN*, LSTM, LSTM empilhada e LSTM bidirecional) para prever a temperatura com um horizonte de 24 horas utilizando o conjunto de dados Jena Climate. Foram consideradas três janelas temporais de entrada (24, 72 e 168 horas). A configuração com melhor desempenho correspondeu a uma LSTM bidirecional com 64 unidades, sem *dropout* e janela de 72 horas, obtendo um RMSLE de 0,06255 no conjunto de teste, equivalente a um RMSE de 2,94 °C e a um MAE de 2,31 °C.

Os resultados demonstram que o *transfer learning* constitui uma abordagem eficaz para tarefas de classificação de imagem, reduzindo simultaneamente o esforço computacional e o tempo de treino. Verificou-se ainda que a janela de 72 horas constitui o compromisso ótimo entre contexto temporal e custo da sequência, e que as células com portas convergem mais rapidamente do que a *SimpleRNN* sem penalizar o erro final.

como arestas e gradientes de cor, enquanto as camadas mais profundas capturam estruturas cada vez mais complexas e semanticamente significativas [1], [2].

Neste trabalho, esta capacidade é explorada num problema de classificação envolvendo 144 variedades distintas de frutos e legumes. Trata-se de uma tarefa particularmente desafiante devido ao elevado número de classes e à forte semelhança visual existente entre algumas categorias, nomeadamente entre diferentes variedades de pera. O melhor modelo desenvolvido alcançou uma *accuracy* de 93,96% no conjunto de teste.

Por outro lado, as redes neurais recorrentes foram concebidas para modelar dependências temporais presentes em dados sequenciais. Entre estas arquiteturas, as redes LSTM destacam-se pela capacidade de atenuar o problema do desaparecimento do gradiente, permitindo capturar relações de longo prazo em séries temporais [1], [2].

A previsão da temperatura atmosférica com antecedência de 24 horas constitui um exemplo representativo deste tipo de problema. A série temporal analisada apresenta ciclos diários e sazonais bem definidos, sendo por isso adequada para avaliar o impacto de diferentes arquiteturas recorrentes e janelas temporais. Os resultados obtidos demonstram que a utilização de contexto temporal mais alargado contribui para melhorar a qualidade das previsões.

I. INTRODUÇÃO

A. Motivação e Contexto

As redes neurais convolucionais representam atualmente uma das abordagens mais relevantes no domínio da visão por computador, devido à sua capacidade para aprender representações hierárquicas de características espaciais. Nas primeiras camadas, estas redes identificam padrões simples,

B. Objetivos

Os objetivos definidos para a componente CNN foram:

- Avaliar o impacto do número de filtros, do *stride*, do *padding* e do tipo de *pooling* no desempenho de classificação;
- Estudar o efeito da aplicação de técnicas de *data augmentation*;

- Identificar situações de *overfitting* e avaliar mecanismos de mitigação através de *Dropout*, regularização L2 e *Early Stopping*;
- Aplicar *transfer learning* recorrendo a um modelo pré-treinado e comparar o seu desempenho com uma CNN desenvolvida de raiz;
- Visualizar os *feature maps* aprendidos pelas camadas convolucionais;
- Analisar exemplos representativos de classificações corretas e incorretas.

Relativamente à componente RNN, os objetivos foram:

- Comparar arquiteturas baseadas em *SimpleRNN*, LSTM simples, LSTM empilhada e LSTM bidirecional;
- Avaliar o impacto de diferentes janelas temporais de entrada (24, 72 e 168 horas) na previsão de temperatura a 24 horas;
- Medir o desempenho utilizando RMSLE, procurando obter valores inferiores ao limiar definido no enunciado.

C. Estrutura do Artigo

A Secção II descreve o problema de classificação de frutos, incluindo o conjunto de dados utilizado, as etapas de pré-processamento, as arquiteturas desenvolvidas, as experiências realizadas e os resultados obtidos.

A Secção III apresenta o problema de previsão meteorológica, abordando a preparação dos dados, as arquiteturas recorrentes consideradas e a avaliação experimental.

Na Secção IV são discutidos e comparados os resultados obtidos, sendo identificadas as principais limitações do trabalho.

Finalmente, a Secção VI apresenta as conclusões e possíveis linhas de trabalho futuro.

II. MODELO CNN – FRUITS-360

A. Descrição do Conjunto de Dados

Foi utilizada a versão *original-size* do conjunto de dados Fruits-360 [3], obtida através da plataforma Kaggle.

As suas principais características são:

- 144 classes correspondentes a diferentes variedades de frutos e legumes;
- 51 345 imagens no conjunto de treino original, posteriormente divididas em 41 076 imagens para treino e 10 269 para validação;
- 25 526 imagens reservadas para teste;
- Imagens RGB com dimensões variáveis;
- Distribuição aproximadamente equilibrada entre classes, embora não perfeitamente uniforme.

As etapas de pré-processamento aplicadas encontram-se resumidas na Tabela I.

TABLE I
PRÉ-PROCESSAMENTO APLICADO AO CONJUNTO FRUITS-360

Passo	Justificação
Redimensionamento para 64×64	Reduz o consumo de memória e o custo computacional, preservando informação visual suficiente para distinguir as diferentes categorias.
Normalização para $[0, 1]$	Facilita o processo de otimização e acelera a convergência durante o treino.
Divisão treino/validação (80/20)	Permite estimar a capacidade de generalização do modelo sem comprometer excessivamente o número de amostras disponíveis para treino.
Baralhar o conjunto de treino	Evita que a ordem física dos dados influencie a composição dos mini-lotes.

B. Metodologia e Arquitetura

A CNN desenvolvida de raiz foi implementada sob a forma de uma arquitetura sequencial composta por três blocos convolucionais. Cada bloco integra uma camada convolucional com filtros de dimensão 3×3 , ativação ReLU, *Batch Normalization* e uma operação de *max-pooling* 2×2 . Após a fase de extração de características, as ativações são convertidas para um vetor unidimensional e processadas por uma camada totalmente ligada responsável pela classificação final.

A configuração que apresentou melhor desempenho durante a fase de experimentação utiliza 32, 64 e 128 filtros nos três blocos convolucionais, respetivamente. O bloco de classificação inclui uma camada densa com 256 neurónios, *Dropout* de 0,4, regularização L2 de 1×10^{-4} e uma camada de saída *softmax* com 144 unidades.

A Tabela II resume a arquitetura final.

TABLE II
ARQUITETURA DA CNN DESENVOLVIDA DE RAIZ

Camada	Saída	Parâmetros
Conv2D (32 filtros)	(64, 64, 32)	896
Batch Normalization	(64, 64, 32)	128
MaxPooling2D	(32, 32, 32)	0
Conv2D (64 filtros)	(32, 32, 64)	18 496
Batch Normalization	(32, 32, 64)	256
MaxPooling2D	(16, 16, 64)	0
Conv2D (128 filtros)	(16, 16, 128)	73 856
Batch Normalization	(16, 16, 128)	512
MaxPooling2D	(8, 8, 128)	0
Flatten	(8192)	0
Dense (256)	(256)	2 097 408
Dropout (0.4)	(256)	0
Dense (144)	(144)	37 008
Total		2 228 560

Durante o treino foram utilizadas as seguintes opções:

- Função de perda: *sparse categorical crossentropy*;
- Otimizador: Adam com taxa de aprendizagem inicial de 10^{-3} ;
- Métrica principal: *accuracy*;
- Tamanho do lote: 64;

- *Early Stopping* com paciência de três épocas;
- *ReduceLROnPlateau* com fator 0,5 e paciência de duas épocas.

1) *Diagrama da Arquitetura*: A Figura 1 apresenta o diagrama da arquitetura gerado através da função `keras.utils.plot_model()`.

Os três blocos convolucionais constituem o módulo responsável pela extração de características, reduzindo gradualmente a resolução espacial das imagens enquanto aumentam a profundidade das representações internas. As camadas densas finais realizam a classificação nas 144 categorias consideradas.

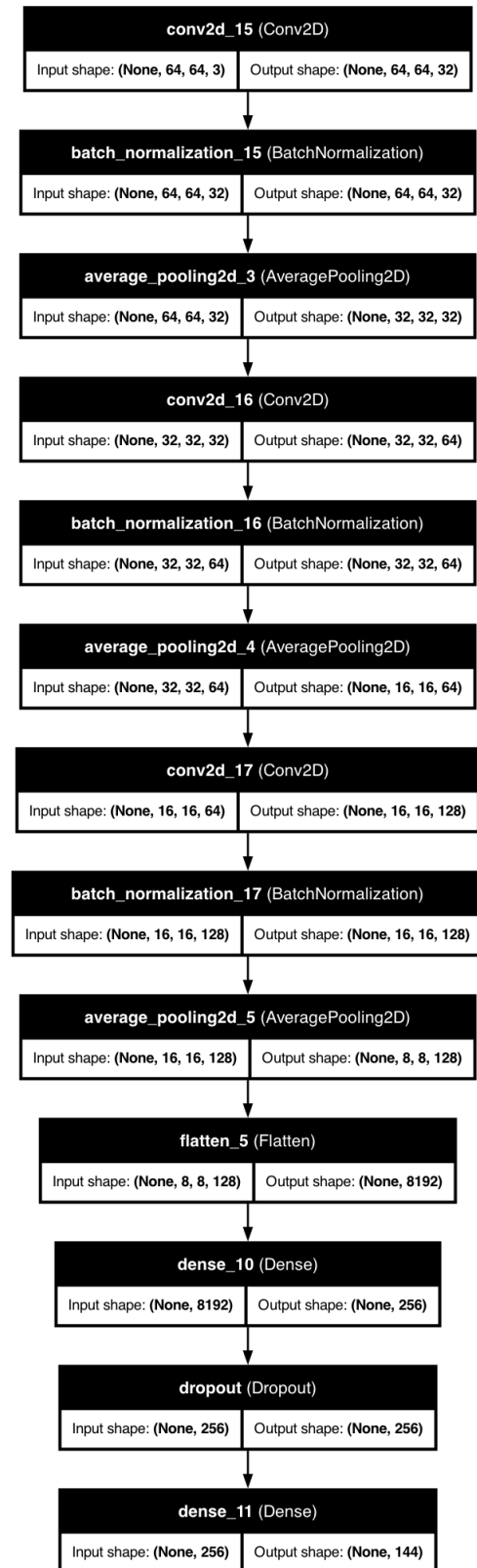


Fig. 1. Arquitetura da CNN desenvolvida de raiz.

C. Experiências Realizadas

Foram avaliadas quatro variantes da arquitetura base, alterando o número de filtros, o tipo de *pooling* e o valor de *stride*. Todas as experiências foram executadas com a mesma semente aleatória e durante cinco épocas.

Os resultados obtidos encontram-se resumidos na Tabela III.

TABLE III
COMPARAÇÃO ENTRE VARIANTES DA CNN

Experiência	Filtros	Stride	Padding	Pooling	Val.
Baseline	32/64/128	1	same	max	0,9985
Menos filtros	16/32/64	1	same	max	0,9966
Avg Pooling	32/64/128	1	same	avg	0,9943
Stride = 2	32/64/128	2	same	max	0,9706

A redução do número de filtros originou uma diminuição ligeira do desempenho, sugerindo que a capacidade adicional fornecida pelos filtros extra é benéfica, embora não determinante para este problema.

A comparação entre *max-pooling* e *average-pooling* revelou vantagem para a primeira abordagem. Este comportamento é coerente com a natureza da tarefa, onde características discriminativas localizadas, como contornos e texturas, desempenham um papel fundamental.

A utilização de convoluções com *stride* igual a 2 provocou a maior degradação da *accuracy*, embora tenha reduzido significativamente o tempo de treino. Este resultado evidencia o compromisso existente entre eficiência computacional e qualidade da representação aprendida.

A melhor configuração foi posteriormente treinada com *Dropout* de 0,4 e regularização L2 de 1×10^{-4} , alcançando uma *accuracy* de validação de 0,9981.

D. Data Augmentation

Foi implementada uma estratégia de *data augmentation* composta por transformações geométricas e fotométricas que preservam a classe da imagem:

- Espelhamento horizontal;
- Rotação aleatória até $\pm 10\%$ de uma volta;
- Zoom até $\pm 10\%$;
- Translação até $\pm 5\%$;
- Alteração de brilho até $\pm 10\%$.

A comparação foi realizada utilizando exatamente a mesma arquitetura, a mesma semente aleatória e o mesmo número de épocas.

Os resultados observados foram:

- Sem *augmentation*: *accuracy* máxima de validação de 0,9791;
- Com *augmentation*: falha total do processo de aprendizagem, resultando em *accuracy* próxima de zero.

A análise dos resultados permitiu identificar a origem do problema. A camada *RandomBrightness* foi aplicada após a normalização das imagens para o intervalo $[0, 1]$, embora o seu comportamento por defeito assumia valores no intervalo $[0, 255]$. Consequentemente, as alterações de brilho

introduzidas tornaram-se excessivas, degradando fortemente a informação visual disponível.

Por este motivo, a experiência não permitiu retirar conclusões válidas relativamente ao impacto real do *data augmentation*. No entanto, evidenciou a importância de garantir coerência entre a normalização dos dados e os parâmetros das transformações aplicadas.

E. Análise de Overfitting

Com o objetivo de demonstrar explicitamente o fenómeno de *overfitting*, foi treinada uma versão deliberadamente sobredimensionada da CNN, utilizando 128, 256 e 512 filtros, sem *Dropout*, sem regularização L2 e sem *Early Stopping*.

Durante o treino observou-se um aumento contínuo da *accuracy* no conjunto de treino, enquanto o desempenho de validação apresentou oscilações significativas. Em simultâneo, a perda de validação começou a divergir da perda de treino após as primeiras épocas.

Este comportamento indica que o modelo começou a memorizar exemplos específicos do conjunto de treino em vez de aprender representações generalizáveis.

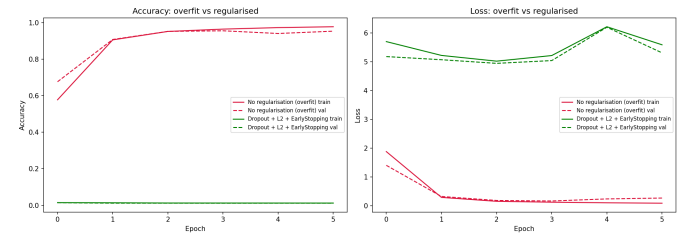


Fig. 2. Comparação entre um modelo com overfitting e um modelo regularizado.

Para mitigar este problema foram adotadas três estratégias:

- *Dropout* de 0,4;
- Regularização L2 de 1×10^{-4} ;
- *Early Stopping* com reposição dos melhores pesos.

A aplicação conjunta destas técnicas permitiu obter um modelo com elevada capacidade de generalização e sem divergência significativa entre treino e validação.

F. Transfer Learning

Para a abordagem baseada em *transfer learning* foi selecionado o modelo MobileNetV2 pré-treinado sobre o conjunto de dados ImageNet, removendo-se a camada de classificação original e substituindo-a por uma nova cabeça adaptada ao problema em estudo.

A escolha deste modelo foi motivada por vários fatores. Em primeiro lugar, o MobileNetV2 apresenta um número reduzido de parâmetros quando comparado com arquiteturas como ResNet50 ou VGG16, tornando-o particularmente adequado para ambientes de treino exclusivamente em CPU. Adicionalmente, suporta imagens de entrada com dimensões reduzidas, compatíveis com o pré-processamento aplicado ao conjunto Fruits-360.

As características aprendidas durante o treino em ImageNet incluem detetores genéricos de arestas, texturas e formas, frequentemente reutilizáveis em problemas distintos de classificação de imagens. Esta propriedade torna o modelo particularmente adequado para tarefas com conjuntos de dados relativamente limitados.

O processo de *fine-tuning* foi dividido em duas fases.

1) *Fase 1 – Base Congelada*: Numa primeira etapa, todas as camadas da base convolucional foram mantidas congeladas e apenas a nova cabeça de classificação foi treinada. Esta cabeça era composta por:

- GlobalAveragePooling2D;
- Dense(256, ReLU);
- Dropout(0.4);
- Dense(144, softmax).

Foram treinados apenas 364 944 parâmetros, obtendo-se uma *accuracy* de validação de 0,9981 após três épocas.

2) *Fase 2 – Fine-Tuning*: Numa segunda fase procedeu-se ao descongelamento das 40 camadas superiores da rede convolucional. A taxa de aprendizagem foi reduzida para 10^{-5} de forma a preservar o conhecimento previamente adquirido durante o treino em ImageNet.

Após o ajuste fino das camadas superiores, a *accuracy* de validação aumentou para 0,9998.

A Tabela IV apresenta a comparação entre a CNN desenvolvida de raiz e o modelo baseado em MobileNetV2.

TABLE IV
COMPARAÇÃO ENTRE CNN PRÓPRIA E MOBILENETV2

Modelo	Val.	Teste	Prec.	Recall	F1
CNN própria	0,9981	—	—	—	—
MobileNetV2	0,9998	0,9396	0,9513	0,9513	0,9513

Embora a diferença entre ambos os modelos seja relativamente reduzida, o MobileNetV2 apresentou o melhor desempenho global, tendo sido selecionado como modelo final.

G. Visualização das Convoluções

Com o objetivo de compreender as características aprendidas pela rede, foram visualizados os primeiros mapas de ativação produzidos pela camada convolucional inicial.

Observa-se que diferentes filtros respondem a diferentes padrões visuais. Alguns destacam contornos, outros enfatizam regiões homogêneas de cor e outros evidenciam transições de intensidade. Este comportamento está de acordo com o esperado para as primeiras camadas de uma CNN, cuja função consiste em extrair características de baixo nível que serão posteriormente combinadas pelas camadas mais profundas.

A Figura 1 ilustra um exemplo representativo dos mapas de ativação obtidos.

H. Melhores e Piores Casos

Os exemplos do conjunto de teste foram ordenados de acordo com a confiança atribuída pelo modelo à classe correta.

Os melhores resultados corresponderam às classes:

- 1) Cantaloupe 3;
- 2) Cactus Fruit Red 1;
- 3) Dates 2.

Todas estas imagens foram classificadas corretamente com confiança próxima de 100%.

Os piores resultados ocorreram em imagens pertencentes à classe Pear 3, frequentemente confundidas com Pear Common 1.

Esta tendência confirma a elevada semelhança visual existente entre determinadas variedades de pera, cujas diferenças subtis se tornam difíceis de distinguir após o redimensionamento das imagens.

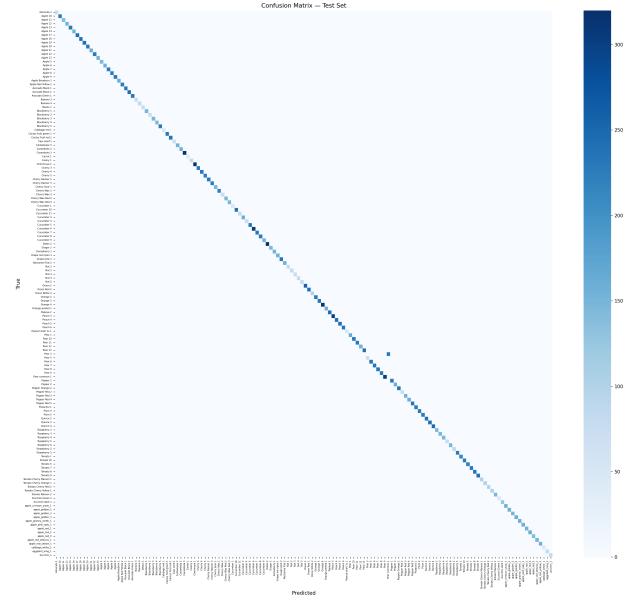


Fig. 3. Matriz de confusão do modelo final.

I. Resultados Finais do Modelo CNN

A Tabela V apresenta os resultados obtidos pelo modelo final no conjunto de teste.

TABLE V
RESULTADOS FINAIS DO MOBILENETV2

Métrica	Valor	Requisito	Cumpre
Accuracy	0,9396	> 0,80	Sim
Precision Macro	0,9513	> 0,70	Sim
Recall Macro	0,9513	> 0,70	Sim
F1 Macro	0,9513	—	Sim

Os resultados demonstram que todos os requisitos quantitativos definidos para a componente CNN foram alcançados com uma margem significativa.

III. MODELO RNN

A. Descrição do Conjunto de Dados

Para a componente de previsão temporal foi utilizado o conjunto de dados Jena Climate, que contém medições me-

teorológicas recolhidas pelo Instituto Max Planck de Bio-geoquímica, localizado em Jena, Alemanha.

As principais características do conjunto de dados são:

- Período temporal entre janeiro de 2009 e janeiro de 2017;
- Frequência original de amostragem de 10 minutos;
- 420 551 observações;
- 14 variáveis meteorológicas;
- Ausência de valores em falta no ficheiro original.

A variável alvo considerada foi a temperatura do ar (T (degC)).

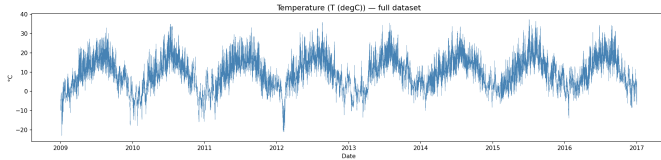


Fig. 4. Série temporal completa da temperatura.

O pré-processamento incluiu:

- Reamostragem para resolução horária;
- Preenchimento de lacunas por *forward-fill*;
- Normalização *z-score*;
- Divisão cronológica dos conjuntos de treino, validação e teste.

Após a reamostragem, o conjunto passou de 420 551 para 70 129 observações.

As entradas foram organizadas sob a forma de janelas deslizantes multivariadas, utilizando as 14 variáveis meteorológicas disponíveis como preditores.

B. Arquiteturas Recorrentes

Foram avaliadas diferentes arquiteturas de redes neurais recorrentes com o objetivo de prever a temperatura atmosférica com um horizonte temporal de 24 horas.

As arquiteturas consideradas foram:

- **SimpleRNN**: arquitetura recorrente básica utilizada como referência;
- **LSTM**: arquitetura baseada em células Long Short-Term Memory, concebida para capturar dependências temporais de longo prazo;
- **Stacked LSTM**: composição de múltiplas camadas LSTM, permitindo aprender representações hierárquicas da série temporal;
- **Bidirectional LSTM**: variante que processa as sequências em ambas as direções durante o treino, aumentando a capacidade de modelação dos padrões temporais.

Todas as arquiteturas utilizaram uma camada densa final responsável pela produção da previsão da temperatura futura.

C. Configuração Experimental

Para avaliar o impacto do contexto temporal disponível para o modelo, foram consideradas três janelas de entrada distintas:

- 24 horas;

- 72 horas;
- 168 horas (7 dias).

A variável alvo corresponde à temperatura observada 24 horas após o instante mais recente presente na janela de entrada.

Durante o treino foram utilizadas as seguintes configurações:

- Otimizador Adam;
- Função de perda Mean Squared Error (MSE);
- Batch size de 64;
- Early Stopping para prevenção de overfitting;
- Monitorização do erro de validação.

O desempenho foi avaliado recorrendo às métricas RMSLE, RMSE e MAE.

D. Comparação das Arquiteturas

Foi realizada uma comparação sistemática entre as diferentes arquiteturas recorrentes e as várias janelas temporais consideradas. O objetivo consistiu em identificar a combinação que proporciona o menor erro de previsão, analisando simultaneamente o impacto da complexidade da arquitetura e da quantidade de contexto temporal fornecido ao modelo.

Os resultados obtidos encontram-se resumidos na Tabela VI.

TABLE VI
MELHORES CONFIGURAÇÕES POR ARQUITETURA (RMSLE DE VALIDAÇÃO)

Modelo	Jan.	RMSLE val.	RMSLE teste	Épocas
SimpleRNN	72h	0,06167	0,06263	14
LSTM	72h	0,06166	0,06195	10
Stacked LSTM	72h	0,06179	0,06232	13
BiLSTM	72h	0,06159	0,06255	7
Modelo final: BiLSTM, 64 unidades, <i>dropout</i> 0,0; RMSE = 2,9371 °C; MAE = 2,3141 °C; 44 609 parâmetros treináveis				

1) *Impacto do Tipo de Camada Recorrente*: Para isolar o efeito do tipo de camada recorrente, comparou-se, para cada arquitetura, a melhor configuração obtida na pesquisa em grelha. Os RMSLE de validação revelaram-se muito próximos entre as quatro arquiteturas: 0,06159 para a BiLSTM, 0,06166 para a LSTM, 0,06167 para a SimpleRNN e 0,06179 para a LSTM empilhada. A diferença entre a melhor e a pior é de apenas 0,0002, pelo que se conclui que, neste problema fortemente periódico, a escolha da célula recorrente tem um efeito marginal sobre o erro final de previsão.

A diferença mais nítida manifesta-se na velocidade de convergência. A SimpleRNN exigiu, em média, 14,7 épocas para convergir (máximo de 34), ao passo que a BiLSTM convergiu em média em 9,4 épocas (máximo de 19) e a LSTM em 10,4 épocas (máximo de 25). Na melhor configuração de cada arquitetura, a SimpleRNN necessitou de 14 épocas, contra apenas 7 da BiLSTM. As células com portas atingem desempenho equivalente em cerca de metade das épocas e com treino mais estável.

Do ponto de vista teórico, esta vantagem decorre do mecanismo de portas das células LSTM. Nas RNN simples, o

gradiente propagado ao longo da sequência é repetidamente multiplicado pelos mesmos pesos recorrentes, originando o desvanecimento (ou explosão) do gradiente e dificultando a aprendizagem de dependências temporais longas [1]. As células LSTM introduzem um estado de célula e portas de esquecimento, de entrada e de saída que regulam o fluxo de informação e mantêm um caminho de gradiente quase constante, permitindo reter contexto ao longo de várias dezenas de passos. A variante bidirecional acrescenta uma segunda passagem em sentido inverso sobre a janela de entrada, enriquecendo a representação do contexto. Conclui-se que, embora as quatro arquiteturas cumpram folgadoamente o limiar exigido, a BiLSTM é a escolha preferível: melhor RMSLE de validação (0,06159), convergência mais rápida (7 épocas) e maior estabilidade de treino.

2) *Impacto das Janelas Temporais*: Para avaliar o impacto da dimensão da janela temporal, fixou-se a melhor arquitetura (BiLSTM) e compararam-se as três janelas. Os RMSLE de validação das melhores configurações foram de 0,06195 para 24 horas, 0,06159 para 72 horas e 0,06261 para 168 horas, com RMSLE de teste de 0,06311, 0,06255 e 0,06447, respetivamente. A janela de 72 horas foi a que produziu o melhor desempenho e constituiu a janela ótima para as quatro arquiteturas testadas.

A superioridade da janela de 72 horas explica-se pelos padrões temporais que consegue captar. Uma janela de 24 horas cobre apenas um ciclo diário completo, sendo suficiente para modelar a periodicidade diurna mas pobre em informação sobre a tendência de curto prazo. A janela de 72 horas inclui três ciclos diários consecutivos, permitindo ao modelo distinguir o padrão diário recorrente da tendência sinótica de vários dias, informação diretamente relevante para a previsão a 24 horas.

Contrariamente ao que seria intuitivamente esperado, o alargamento para 168 horas (1 semana) não melhorou os resultados — pelo contrário, degradou ligeiramente o RMSLE de validação (0,06261 contra 0,06159). Tal sugere um efeito de retornos decrescentes: a informação útil para um horizonte de 24 horas concentra-se nos últimos dias, e o contexto adicional de uma semana introduz sobretudo ruído e padrões pouco informativos, ao mesmo tempo que produz sequências mais longas e mais difíceis de otimizar. A análise do *gap* entre a perda de validação e a de treino confirma esta leitura: o *gap* foi de 0,0557 (24 h), 0,0706 (72 h) e 0,0290 (168 h); a janela de 168 horas apresentou o menor *gap* (melhor generalização relativa), enquanto a de 72 horas, apesar do melhor RMSLE absoluto, registou o maior *gap*. A paragem antecipada atuou em todas as configurações (8, 7 e 7 épocas, respetivamente), contendo o sobreajuste. Conclui-se que a janela de 72 horas constitui o compromisso ótimo entre contexto temporal suficiente e custo/ruído da sequência.

3) *Análise das Curvas de Aprendizagem*: A análise das curvas de aprendizagem ao longo das 72 configurações evidencia diferenças sistemáticas de convergência entre arquiteturas. A *SimpleRNN* foi a mais lenta, exigindo em média 14,7 épocas e até 34 épocas em algumas configurações; a LSTM empilhada

convergiu em média em 12,7 épocas (máximo de 29), a LSTM em 10,4 épocas (máximo de 25), e a BiLSTM foi a mais eficiente, com média de 9,4 épocas e máximo de 19. Dado que os RMSLE finais são praticamente equivalentes entre arquiteturas, esta diferença de épocas traduz diretamente a maior eficiência de otimização das células com portas — consistente com o caminho de gradiente mais estável que evita o desvanecimento característico das RNN simples.

O comportamento da perda de validação face à de treino revela um sobreajuste contido em todas as configurações. A paragem antecipada (paciência 5, com reposição dos melhores pesos) interrompeu todos os treinos muito antes do limite de 50 épocas — tipicamente entre 7 e 14 épocas nas melhores configurações —, indicando que os modelos atingem rapidamente o seu ponto de melhor generalização. Conclui-se que a combinação de células com portas e paragem antecipada assegura uma convergência rápida e estável, sem necessidade de regularização adicional agressiva.

E. Resultados

Todas as 72 configurações cumpriram com larga margem o limiar do enunciado ($\text{RMSLE} < 1$): os valores de teste situaram-se no intervalo estreito de 0,06193 a 0,06804 e os de validação entre 0,06159 e 0,06566. A concentração de todas as configurações numa banda tão estreita admite uma dupla leitura: por um lado, reflete a natureza fortemente periódica e previsível da série de temperatura, em que mesmo arquiteturas simples capturam o essencial do sinal; por outro, decorre parcialmente da escala da métrica, uma vez que o RMSLE foi calculado sobre valores normalizados (*z-score*) deslocados por um desvio constante de 4,6742, o que comprime fortemente as diferenças no espaço logarítmico. Em consequência, as métricas em unidades físicas — RMSE de 2,94 °C e MAE de 2,31 °C — são indicadores mais interpretáveis do desempenho real.

Observou-se ainda que a ordenação por validação não coincide perfeitamente com a de teste: a configuração com menor RMSLE de teste (LSTM, 168 h, 64 unidades, sem *dropout*: 0,06193) não é a de menor RMSLE de validação (BiLSTM, 72 h: 0,06159). Esta divergência resulta de os conjuntos de validação e de teste serem segmentos cronológicos distintos e de as diferenças entre modelos serem da ordem do ruído entre execuções. Uma vez que a seleção de modelos tem obrigatoriamente de assentar apenas no conjunto de validação, adotou-se a BiLSTM de 72 horas. A melhor configuração — BiLSTM, janela de 72 horas, 64 unidades e *dropout* 0,0 — traduz escolhas justificáveis: a janela de 72 horas pelo compromisso contexto/ruído (Secção III-D2); 64 unidades por proporcionarem capacidade suficiente sem sobreajuste; e a ausência de *dropout* porque a paragem antecipada e o treino curto (7 épocas) já asseguram regularização adequada.

O melhor modelo obtido foi posteriormente utilizado para gerar previsões no conjunto de teste.

F. Análise dos Melhores e Piores Casos

Nos melhores casos, o erro absoluto de previsão foi praticamente nulo ($\approx 0,00^\circ\text{C}$), tendo o modelo previsto $5,6^\circ\text{C}$, $6,1^\circ\text{C}$ e $9,0^\circ\text{C}$, coincidentes com os valores reais. Estas janelas correspondem a períodos amenos e meteorologicamente estáveis, com ciclos diários regulares e sem transições abruptas. Nestas condições, a BiLSTM explora plenamente o que aprendeu — a estrutura periódica diária e a continuidade suave da temperatura ao longo de poucos dias —, extrapolando o padrão recente com elevada precisão.

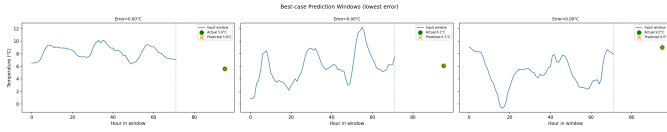


Fig. 5. Comparação entre valores reais e previstos pelo melhor modelo (BiLSTM, 72 h).

Nos piores casos, o erro ascendeu a $10,36^\circ\text{C}$, $10,99^\circ\text{C}$ e $11,03^\circ\text{C}$: em todas estas janelas, a série de entrada termina em valores amenos ($\sim 9\text{--}10^\circ\text{C}$), mas a temperatura real 24 horas depois desce bruscamente para perto de 0°C ($-0,6^\circ\text{C}$, $-0,7^\circ\text{C}$ e $-0,1^\circ\text{C}$), tendo o modelo previsto $9,8^\circ\text{C}$, $10,3^\circ\text{C}$ e $10,9^\circ\text{C}$. O fenómeno subjacente é a passagem rápida de uma frente fria — um evento raro, abrupto e não anunciado pelo padrão recente da janela. Como o modelo foi treinado para minimizar o MSE, aprende a prever o valor condicional médio, favorecendo a continuação suave do padrão observado e penalizando pouco a falha em transições extremas e pouco frequentes.



Fig. 6. Comparação entre valores reais e previstos pelo pior modelo (BiLSTM, 72 h).

Abordagens alternativas poderiam mitigar esta limitação: a previsão probabilística (por exemplo, com perdas baseadas em quantis) quantificaria a incerteza e sinalizaria a possibilidade de descidas acentuadas; e mecanismos de atenção permitiriam ao modelo ponderar seletivamente indícios precursores presentes nas restantes variáveis da janela. Esta limitação é análoga à observada no problema CNN, em que os piores casos se concentraram nas sete variedades de pera sistematicamente confundidas entre si por elevada semelhança visual (F1 de 0,0), apesar de uma exatidão global de 93,96%. Em ambos os problemas, o erro residual não é aleatório, mas concentra-se nos casos intrinsecamente mais ambíguos ou raros.

IV. CONCLUSÕES

Neste trabalho foram estudadas duas aplicações distintas da aprendizagem profunda: classificação de imagens através de CNN e previsão de séries temporais utilizando RNN.

Na componente de visão por computador, o modelo MobileNetV2 ajustado através de fine-tuning apresentou o melhor desempenho, ultrapassando os requisitos definidos para accuracy, precision e recall.

Na componente de previsão temporal, as arquiteturas LSTM revelaram-se mais adequadas para a modelação da série Jena Climate, evidenciando a importância da memória de longo prazo em tarefas deste tipo.

Como trabalho futuro poderão ser exploradas arquiteturas mais avançadas, incluindo Transformers para séries temporais e modelos de visão mais recentes para classificação de imagens.

REFERENCES

- [1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.
- [2] F. Chollet, *Deep Learning with Python*, 2nd ed. Shelter Island, NY: Manning, 2021.
- [3] H. Muresan and M. Oltean, “Fruit recognition from images using deep learning,” *Acta Univ. Sapientiae, Informatica*, vol. 10, no. 1, pp. 26–42, 2018.